



ADDING ROLLUP THREADS TO YOUR EXPERIENCE

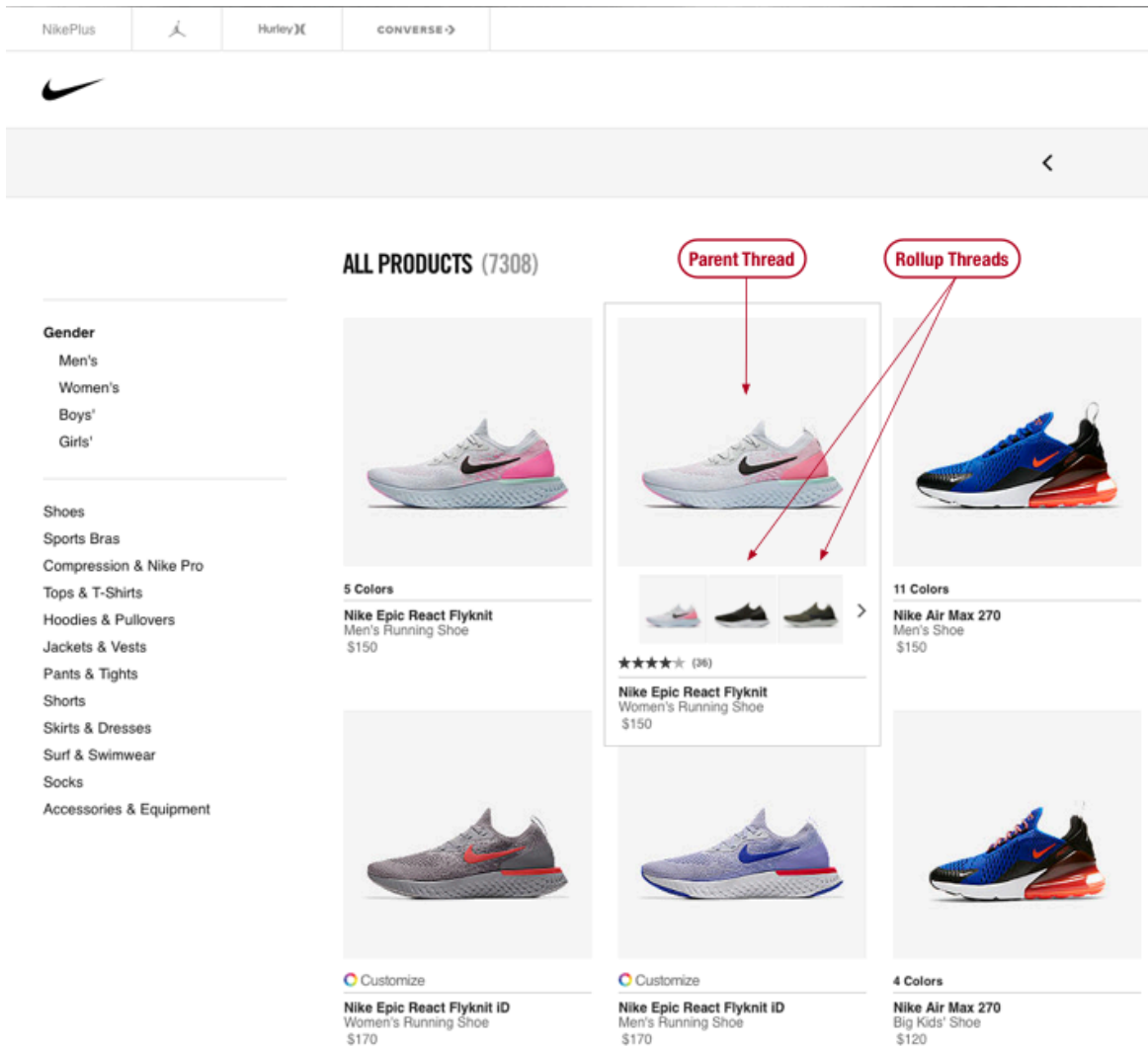
Last Updated: 01/28/2019

Use the [Product Feed Rollup Threads API \(https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api \)](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api) to get related product content for your digital experience, for example to show consumers a grid wall of Nike products.

TIP: Before using this guide, first read the [Product Feeds Developer's Guide \(https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html \)](https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html) to understand the basics about Threads.

What is a Rollup Thread?

Remember that a Thread contains product content and information. A Rollup Thread is a Thread that is related to, and nested within, a parent Thread. For example, a Thread containing content for a particular shoe color might have seven Rollup Threads nested within it. In this case, each Rollup Thread contains content for the other available colors of that shoe. Using Rollup Threads makes it simpler for you to build a product grid wall experience like this:



Get a List of Rollup Threads

- ✓ **Product Grid Wall:** Get product threads along with related 'rollup' threads
- ✓ **Get customized search results**

To get a list of Threads, with their associated Rollup Threads, execute a request to the [Rollup Threads List \(https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api#rollup-threads-rollup-threads-list-get \)](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api#rollup-threads-rollup-threads-list-get) endpoint.

Use Case	Sample Query
Threads for a taxonomy ID	<code>https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=marketplace(US)&filter=language(en)&filter=taxonomyIds(c2228131-f12b-4513-84cd-55ae15d6723d)</code>
Search all on Nike.com products in US by “Men’s Jordan”, sorted by newest first	<code>https://api.nike.com/product_feed/rollup_threads/v2?consumerChannelId=d9a5bc42-4b9c-4976-858a-f159cf99c647&filter=language(en)&filter=marketplace(US)&searchTerms=Men’s%20Jordan&sort=effectiveStartSellDateDesc</code>

TIP: For sample responses, see the [Rollup Threads API Reference](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api) (<https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api>).

How Can I Control the Rollup?

By default, the relationship between the Rollup Thread(s) to the parent Thread is based upon a rollup type and rollup key defined in Nike’s product information system. As the client of the API, you do not have direct control over those values.

Both of these fields are located in the API response in the

productInfo.merchProduct.productRollup object, field names **type** and **key**.

- rollup **type**: a group of products related by something, e.g. colors associated with a style number, or shoes of the same width. Example values are “Standard”,

“WidthGroup”, “NFL”, or “NBA”.

- rollup **key**: a specific instance of a rollup type, e.g. a value representing a particular style number rollup. Example value is “xqTPKlqE”.

How to Get Only the Threads You Need

If there are more threads in the API response than you want, add more specific identifiers to the request as follows.

1. Filters

Use any of the [supported filter query parameters \(https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api \)](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api) to get more specific results in the response. For example, to select one or more rollup keys, append `filter=rollupKey(0cc4QN)` to the request URI.

2. Sorting

Sort the results using any of the [supported sort query parameters \(https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api \)](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api). For example, to sort by the current price in ascending order, append query parameter `sort=productInfo.merchPrice.currentPriceAsc` to the request URI.

TIP: The [Product Feeds API Reference \(https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api \)](https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api) is the source of truth for all supported query parameters.

Custom Search Rules

You also have the ability to configure additional rules in the Apollo search administration tool that will **override the default rollup behavior described above, exclusively for your**

consumerChannelId.

For example, if you want Rollup Threads that are related by something other than style number (i.e. “type”: “Standard”), then you would create search rules in Apollo based on how you want the rollup to function. You could also create a rule in Apollo to exclude customized Nike ID products or gift cards from your results, if desired.

Ultimately, Apollo is where you can control how the Rollup Threads are returned to you in the response from this API.

Consumer Channel ID Versus Channel ID

Your Consumer Channel ID is unique to your app and allows you to have custom search rules to return only the parent and Rollup Threads that you need. But how is Consumer Channel ID related to the Channel ID you might be using with Product Feeds API?

Consumer Channel ID and Channel ID are not directly related and therefore should not be used interchangeably. Both are unique IDs that help you to get only the data you need from each respective API.

When calling the Product Feed Rollup Threads API, Consumer Channel ID is required and Channel ID is not allowed.

NOTE: Threads returned by the Product Feed Rollup Threads API are pre-filtered for the Nike.com Channel ID.

I Already Use Product Feeds v2. How is This API Response Different?

The following diagram describes the how the structure of the response from the Rollup Threads API differs from Product Feeds:



Rollup Threads API

Response Structure Compared to Feeds v2



Feeds v2: Threads List

GET /product_feed/threads/v2

```
{
  "pages": {
    "objects": [
      {
        Thread 1
      },
      {
        Thread 2
      }
    ]
  }
}
```

Rollup Threads List

GET /product_feed/rollup_threads/v2

```
{
  "pages": {
    "objects": [
      {
        Thread 1 {
          "rollup": [
            {
              Related Thread A
            },
            {
              Related Thread B
            },
          ],
        },
        Thread 2 {
          "rollup": [
            {
              Related Thread X
            },
            {
              Related Thread Y
            },
          ],
        }
      ]
    ]
  }
}
```

"rollup" is an array of threads related to the parent thread

NOTE:

The structure of the responses has been simplified to highlight the differences. Refer to each API.md for the detailed schemas.

API Quick Reference

HTTP Verb	Endpoint Name	Endpoint Description	URI Format
GET	Rollup Threads	Get a product Thread with related Threads nested within (https://developer.niketech.com/docs/projects/Product%20Feed%20Rollup%20Threads%20Service%20API%20V2?tab=api)	/product_feed/rollup_threads/ v2{?filter,anchor,count,sort,searchTerms,rollupCount,rollupField,consumerChannelId}

Best Practices

See the Best Practices section of the [Product Feeds Developer's Guide \(https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html#best-practices \)](https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html#best-practices).

Troubleshooting

I'm getting a 200 response, but the Thread data and/or Rollup Threads are not as expected

- Check your Smart Search rules configuration in the Apollo application to ensure that the rules are correct.
- Check the rollup key & type from Prodigy for the Parent Thread is as expected.
- Reach out to Product Feeds team on Slack for assistance: [#nde-product-feeds \(https://nikedigital.slack.com/messages/CAPF62A66 \)](https://nikedigital.slack.com/messages/CAPF62A66)

TIP: See the Troubleshooting section of the [Product Feeds Developer's Guide](https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html#troubleshooting) (<https://developer.niketech.com/nde-docs/doc/commerce/product/use-product-feeds.html#troubleshooting>) for more general troubleshooting information.

Terms of Service

It is recommended that you send a caller ID header in every request to this API to help troubleshoot unexpected responses. See the Registration section of [Using NDe APIs](https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#registration) (<https://developer.niketech.com/nde-docs/doc/getting-started/using-nike-apis.html#registration>) on how to create and register your caller ID.

Authentication

No authentication or authorization is required to use this API.

Prerequisites

In order to use the Product Feed Rollup Threads v2 API, you need to:

- **Obtain a Consumer Channel ID – REQUIRED**

Fill out a request form (<https://confluence.nike.com/display/G11N/Request+Form+for+a+new+Consumer+Channel>) to define your needs for a Consumer Channel ID. Once submitted, this form will be used to assess whether an existing ID can be used or a new ID needs to be created. This is different from the Channel ID you may be using to call the Product Feeds endpoint. See [Consumer Channel ID and Channel ID](#) for a comparison between the two ID types.

- **Configure Custom Search Rules – OPTIONAL**

The default key used for rolling up Threads is **productInfo.merchProduct.productRollup.key**. If you require any custom search rules to refine how Threads are rolled up, you can work with the Search team to create them.

Contact the [Apollo Product Owner](#) for assistance.

Glossary

See the [Glossary \(https://developer.niketech.com/nde-docs/doc/commerce/reference/glossary.html \)](https://developer.niketech.com/nde-docs/doc/commerce/reference/glossary.html).

Document Change Log

Summary	Date	Description
Initial draft	05/17/2018	Initial Draft
Update	07/19/2018	Updated how to obtain a consumerChannelId

Related Links

[NDe Docs Home](#)

[Get Started \(https://developer.niketech.com/nde-docs/doc/getting-started/get-started.html \)](https://developer.niketech.com/nde-docs/doc/getting-started/get-started.html)